AI QA ENGINEER

Multi-Agent AI-Powered Automated Code Quality & Testing Platform

College Final Year Project Presentation & Viva Guide

Project Domain: Artificial Intelligence & Software Engineering	**Architecture:** LangGraph Multi-Agent State Machine
LLM Model: Google Gemini 2.5 Flash	**Backend:** Python 3.11+ / FastAPI / Pytest

SLIDE 1: TITLE & INTRODUCTION

- **Project Name:** AI QA ENGINEER — Multi-Agent Software Quality & Testing Platform
- **Domain:** Applied Artificial Intelligence, Software Engineering & QA Automation
- **Goal:** Simulate an entire autonomous QA team using specialized collaborative AI agents.

■■ ***SPEAKER NOTES (Say This):*** *Good morning respected guide and coordinator. Today I present 'AI QA Engineer', an autonomous multi-agent platform that automates full-lifecycle code quality inspection, test generation, test execution, security auditing, and root-cause analysis.*

SLIDE 2: PROBLEM STATEMENT

- **Manual QA Bottleneck:** Writing comprehensive unit tests covering edge cases is time-intensive and error-prone.
- **Shallow Happy-Path Testing:** Developers often test only expected inputs, missing zero-division, type mismatches, and boundary crashes.
- **Security Blindspots:** Vulnerabilities like SQL injections and hardcoded secrets routinely leak into production.
- **Fragmented Tooling:** Linters, security scanners, and test runners output disjointed logs without root-cause reasoning.

■■ ***SPEAKER NOTES (Say This):*** *Software testing is currently fragmented. Developers juggle multiple tools without unified synthesis, and manually triaging failed tests consumes hours of valuable engineering time. Our system solves this by introducing automated multi-agent reasoning.*

SLIDE 3: EXISTING VS. PROPOSED SYSTEM

- **Key Paradigm Shift:** From fragmented manual scripting to a unified, stateful multi-agent orchestrator.
- **Zero-Hallucination Policy:** Empirical metrics are strictly measured with deterministic tools, while LLMs handle reasoning.

Dimension	Existing Manual QA	AI QA Engineer (Proposed)
Test Authoring	Manual, slow, developer-dependent	Automated synthesis via Gemini 2.5 Flash
Security Audit	Separate manual CLI tools	Integrated Bandit AST & Heuristic scanning
Defect Diagnosis	Manual stack trace debugging	Automated Root Cause Analysis (RCA) with fixes
Coordination	Ad-hoc manual workflows	Stateful LangGraph multi-agent pipeline

■■ ***SPEAKER NOTES (Say This):*** *Traditional testing requires human manual intervention at every phase. Our proposed AI QA Engineer uses a state machine to orchestrate 10 specialized agents, strictly separating empirical measurements from generative AI reasoning.*

SLIDE 4: SYSTEM ARCHITECTURE & DATA FLOW

- **Presentation Tier:** Responsive Web Dashboard (HTML5 / CSS3 / Vanilla JS) with real-time state polling.
- **Backend API Tier:** FastAPI asynchronous REST backend with background task execution.
- **Orchestration Tier:** LangGraph StateGraph managing sequential agent activations across a shared QAState.
- **Verification Engine:** Subprocess execution of Pytest, pytest-cov, Ruff, and Bandit.

■■ ***SPEAKER NOTES (Say This):*** *Our architecture follows a clean 4-tier design. When the user selects a project, FastAPI triggers our LangGraph state machine. Each agent mutates a shared state dictionary in memory, driving both deterministic tools and Gemini API calls.*

SLIDE 5: MULTI-AGENT CATALOG (10 SPECIALIZED AGENTS)

- **1. Supervisor Agent:** Initializes the state machine, validates inputs, logs timestamps, and coordinates execution.
- **2. Code Loader Agent:** Recursively scans files, discovers Python source files, and builds project structure.
- **3. Code Analyzer Agent:** Runs Ruff linter and computes AST cyclomatic complexity and code smells.
- **4. Security Agent:** Executes Bandit to detect SQL injections, command injections, and hardcoded API keys.
- **5. Test Generator Agent:** Prompts Gemini to synthesize comprehensive Pytest test cases covering failure modes.
- **6. Test Runner Agent:** Executes generated tests in an isolated subprocess and captures pass/fail counts.
- **7. Coverage Agent:** Runs pytest-cov to calculate statement and line-by-line coverage percentages.
- **8. Bug Analysis Agent:** Correlates failed tests and static issues to categorize defects by severity.
- **9. Root Cause Agent (RCA):** Diagnoses WHY defects happened, business risks, and code-level fixes.
- **10. QA Report Agent:** Computes Overall Quality (0-100), Security Score, and writes executive summaries.

■■ ***SPEAKER NOTES (Say This):*** *Instead of using a single monolithic prompt, we distribute responsibilities across 10 specialized agents. This modular design prevents context loss, enables isolated retries, and guarantees deterministic accuracy.*

SLIDE 6: TECHNOLOGY STACK

- **Core Programming:** Python 3.11+ (TypedDict, AST parser, Subprocess)
- **Backend Framework:** FastAPI, Uvicorn, Pydantic v2
- **AI & Orchestration:** Google Gemini 2.5 Flash (google-genai SDK), LangGraph, LangChain Core
- **Testing & Security:** Pytest, Pytest-cov, Ruff, Bandit
- **Frontend UI:** Modern Dark Glassmorphic Dashboard (HTML5, CSS3, Vanilla JavaScript)

■■ ***SPEAKER NOTES (Say This):*** *Our technology stack combines high-performance Python asynchronous APIs with LangGraph state orchestration and Google's Gemini 2.5 Flash model for natural language reasoning and code generation.*

SLIDE 7: DEMO CASE STUDY & PLANTED DEFECTS (DEMO_PROJECT/)

- **calculator.py:** Zero-division error in divide(), missing upper-bound in calculate_discount(), empty list in average().
- **user_service.py:** Hardcoded SECRET_KEY and API_KEY, plaintext password storage, weak MD5 hashing.
- **payment.py:** Command injection via subprocess(shell=True), unvalidated negative payment amounts.
- **database.py:** Direct f-string SQL injection vulnerabilities in get_user() and create_user().
- **utils.py:** Arbitrary code execution via pickle.load() and eval() on unsanitized inputs.

■■ ***SPEAKER NOTES (Say This):*** *To validate our platform, we created demo_project/ containing realistic defects. In our live test run, the platform successfully identified all security vulnerabilities, exposed logic bugs through generated tests, and diagnosed the root causes.*

SLIDE 8: ROOT CAUSE ANALYSIS (RCA) DEEP DIVE

- **Why It Happened:** Explains the fundamental reason (e.g., missing input validation guard clause).
- **Affected Component:** Pinpoints exact module and function (e.g., calculator.py -> divide()).
- **Business & Technical Impact:** Evaluates production outage risks and unhandled exception propagation.
- **Actionable Fix & Confidence:** Provides drop-in corrected code with AI confidence percentage (e.g., 96%).

■■ ***SPEAKER NOTES (Say This):*** *The most powerful feature of AI QA Engineer is Root Cause Analysis. Rather than simply alerting that a test failed, our RCA agent explains why it failed, what business risk it poses, and generates a concrete Python code snippet to fix it.*

SLIDE 9: SCORING FORMULA & HUMAN-IN-THE-LOOP SIGN-OFF

- **Quality Score (0-100):** Weighted: Code Quality (30%) + Test Pass Rate (25%) + Coverage (20%) - Bug Severity Penalties (25%).
- **Security Score (0-100):** Deductions based on CVSS severity: Critical (-20 pts), High (-10 pts), Medium (-5 pts).
- **Human-in-the-Loop (HITL):** Interactive sign-off where QA leads review findings and mark reports as Approved or Rejected with audit notes.

■■ ***SPEAKER NOTES (Say This):*** *We compute an Overall Quality Score and Security Score using a transparent, multi-metric formula. The platform includes a Human-in-the-Loop sign-off interface, allowing QA leads to approve releases or request re-analysis.*

SLIDE 10: ADVANTAGES, FUTURE SCOPE & CONCLUSION

- **Key Advantages:** Zero metric hallucination, 80%+ reduction in test authoring time, unified security and QA visibility.
- **Future Scope:** Support for Java and TypeScript, native GitHub Actions CI/CD integration, and automated PR fix creation.
- **Conclusion:** AI QA Engineer demonstrates that combining generative AI with deterministic testing tools creates a reliable, autonomous QA platform.

■■ ***SPEAKER NOTES (Say This):*** *In conclusion, AI QA Engineer proves that multi-agent architectures paired with deterministic tools can automate the complex cognitive workflow of software testing. Thank you, and I now welcome any questions.*

TOP 10 VIVA QUESTIONS & ANSWERS (CHEAT SHEET)

Q: 1. Why use LangGraph instead of a single LLM prompt?
A: A single prompt cannot reliably execute code, calculate coverage, run security audits, and write reports without hallucinating numbers or losing context. LangGraph provides a stateful state machine coordinating specialized agents.

Q: 2. How do you prevent LLM hallucinations?
A: We strictly separate empirical measurement (Pytest for pass/fail, pytest-cov for coverage, Bandit for security) from generative reasoning (Gemini for test synthesis, RCA, and summary writing).

Q: 3. What is the role of the Supervisor Agent?
A: The Supervisor initializes the execution graph, logs timestamps, tracks agent states (Pending -> Running -> Completed), and manages error boundaries.

Q: 4. How does shared state work?
A: We use a TypedDict called QAState. As LangGraph transitions between nodes, each agent reads its inputs and updates its specific output keys in memory.

Q: 5. What happens if Gemini API is offline?
A: DEMO_MODE=true automatically activates, using deterministic AST checks, mock test suites, and pre-computed AI diagnostics so the application never crashes.

Q: 6. How are unit tests generated?
A: The Test Generator Agent feeds source files into Gemini with a specialized test-engineering prompt that enforces edge-case, boundary, and exception testing using pytest.raises().

Q: 7. How does the Security Agent work?
A: It runs Bandit, an AST-based security linter that matches syntax nodes against vulnerability rules (B602 command injection, B307 eval, B105 hardcoded secrets).

Q: 8. How is code coverage measured?
A: The Coverage Agent invokes pytest --cov in a subprocess, parsing XML line coverage data to compute statement coverage percentages per file.

Q: 9. What is Human-in-the-Loop governance?
A: Before a report is finalized, a human QA lead reviews the findings on the dashboard and officially approves or rejects the release with audit notes.

Q: 10. How can this project scale?
A: By integrating Celery and Redis for asynchronous task queues, PostgreSQL for report storage, Docker for sandboxed test execution, and GitHub Actions for CI/CD gating.